실전 1 보고서 1부 ④ 탐지기 선정 근거

> - **과제**: 금융사 사내 문서에서 개인정보 7종을 찾는 탐지기를 규칙 기반·토큰 분류·LLM 세 방식으로 비교하고, 운영에 쓸 탐지기를 근거 있게 고른다.
> - **데이터**: 제공 `pii_synth_finance` dev 100건(정답 548개) · test 200건(정답 1,165개) / 추가 시험 데이터: 엣지 케이스 64건(정답 116개), 일상 문서 1,983건(정답 3,375개) / **멘토 holdout 187건(정답 912개)**
> - **채점**: 멘토 제공 `run_compare.py` (strict·partial 정밀도/재현율/F1, mask_rec, 라벨별·어려운 사례 재현율, 일관성)
> - **작성**: 2026-10-23, 수치 갱신 2026-10-26 (탐지기 v8) · 저장소 `hosung` 브랜치 `실전프로젝트1/` · 작업 기록 전체는 `worklog.md`

보고서 구성: 0 요약 및 결론 · **1부** ① 데이터 관찰 리포트 · ② 방식별 비교표 · ③ 오류 유형 분석 · ④ 탐지기 선정 근거 · **2부** ⑤ Holdout 데이터 관찰 · ⑥ Holdout 성능 · ⑦ Holdout 오류 분석 · ⑧ Holdout 개선 내역 · 부록 개선 과정과 데이터 사용 기록 (`report/` 폴더)

> **1부 수치 기준**: holdout 오류 분석 후 고친 **탐지기 v8**의 값이다. 이후 v9(부록 A-4)에서 dev·test·엣지 결과는 바뀌지 않았고, 일상 문서 평가용만 조금 올랐다(규칙 + 토큰 strict F1 0.956 → 0.961, mask_rec 0.988 → 0.989). holdout을 보기 전(v7) 값과 개선 전후 비교는 ⑥ Holdout 성능·⑧ Holdout 개선 내역 참고. 1부의 '일상 문서'는 팀이 만든 생성 문서(개선용 synth_dev / 평가용 synth_holdout)이고, 멘토가 준 holdout과 다르다.

4-1. 결론

- **운영 탐지기: 규칙 기반 + 토큰 분류(atonlee) 전체 합집합**
- **LLM: 탐지에는 쓰지 않고 RAG 답변 생성에서만 사용 (마스킹된 텍스트만 전달)**

4-2. 기준별 판단

기준	규칙 단독	토큰 단독	규칙 + 토큰	LLM 단독	규칙 + LLM
놓치지 않기: test mask_rec	0.997	0.989	**1.000**	0.988 (gpt-4.1-mini)	1.000 (gpt-5.5)
놓치지 않기: 처음 보는 문서 mask_rec (일상 문서)	0.972	0.983	**0.988**	미측정	미측정
놓치지 않기: 멘토 holdout mask_rec (개선 전)	0.961	0.979	**0.993**	미측정	미측정
정밀도 하한선 0.90 (test / 일상 문서)	0.998 / 0.957	0.958 / 0.931	**0.976 / 0.934**	0.996 / —	0.997 / —
속도 (ms/문서, test)	9	53 (부하 시 ~100)	**62 (부하 시 ~112)**	1,292	1,781
일관성	100%	100%	**100%**	77.5~98.5%	88~100%
원문 외부 전송	없음	없음	**없음**	있음	있음
CPU 서버 운영	가능	가능	**가능**	API 의존	API 의존
라이선스	Kiwi(kiwipiepy) Apache-2.0	atonlee Apache-2.0	**둘 다 Apache-2.0**	상용 API	상용 API

4-3. 라벨별로 방식을 나눠 쓰는 조합 검토

라벨마다 강한 방식이 다르다: 정형 라벨(전화·계좌·카드)은 규칙이, 이름은 토큰이 강하다(② 방식별 비교표 2-5). 그래서 두 가지 조합을 비교했다.

조합	test mask_rec	일상 문서 mask_rec	일상 문서 strict P
전체 합집합 (모든 라벨을 합침, 겹치면 규칙 우선)	1.000	**0.988**	0.934
이름만 결합 (정형 라벨은 규칙만, 이름만 규칙 ∪ 토큰)	1.000	0.976	0.953

이름만 결합하면 정밀도는 오르지만, 규칙이 놓친 번호·주소를 토큰이 잡아도 버리게 되어 처음 보는 문서에서 mask_rec이 0.012 떨어진다(일상 문서에서 주민번호 재현율 규칙 0.939 vs 합집합 0.988). 제공 데이터에서는 규칙의 정형 라벨이 이미 완벽해 두 조합이 같은 점수라 차이가 보이지 않았다. **재현율 우선 기준에 따라 전체 합집합을 선택**했다.

4-4. 탈락시킨 방식과 이유

방식	탈락 이유 (수치)
규칙 단독	가장 빠르고 제공 데이터에서 가장 정확하지만, 처음 보는 문서에서 2.8%를 놓침(mask_rec 0.972), 멘토 holdout(개선 전)에서는 3.9%(0.961, 외자 이름 재현율 0.289). dev 문구에 기댄 규칙을 고치면 test 재현율이 약 8%p 떨어짐 → 템플릿이 다른 문서에서 무너질 위험
토큰 단독	엣지 케이스 mask_rec 0.920(안심번호·휴대폰 계좌·외국인등록번호 미탐), 경계가 부정확해 strict F1 0.962, 처음 보는 문서 계좌 재현율 0.731, 멘토 holdout 계좌 형식 변형 재현율 0.696
다른 토큰 모델	veil은 mask_rec이 비슷하나 5배 느림, dlp는 22배 느림, seongyeon은 경계 오류가 많고 CC-BY-SA-4.0(사내 배포 시 검토 필요), 기준 모델 bccard는 재현율 0.773
LLM 단독	가장 좋은 gpt-5.4-mini도 일관성 81%·미끼 오탐 9. 클로바 HCX-007은 회사 이름을 사람으로 잡고 가장 느리며(2,629ms) 일관성 77.5%, DASH-002는 직원끼리 쓴 문서 32건을 통째로 "개인정보 없음"(mask_rec 0.667). 모두 원문 외부 전송 필요
규칙 + LLM	test 정확도는 가장 높으나(규칙 + gpt-5.5: mask_rec 1.000, 정밀도 0.997) **재현율 이득이 없음**(규칙 + 토큰도 mask_rec 1.000). 문서당 1,781ms로 **약 16~29배 느림**(1만 건 약 4.9시간 vs 약 10~19분), 원문을 외부로 보내야 하고, 문서마다 API 비용

4-5. 운영 조건 정리

- **원문 외부 전송 없음**: 규칙과 토큰 모델 모두 사내 CPU 서버에서 돌아간다. 개인정보를 찾으려고 원문을 클라우드 LLM에 보내는 모순이 없다.
- **CPU 환경**: GPU 없이 문서당 약 0.06~0.11초. 1만 건 약 10~19분(PC 부하에 따라).
- **라이선스**: Kiwi(kiwipiepy) Apache-2.0, atonlee Apache-2.0. 학습 데이터(`BCCard/pii-masking-openpii-finance`, `townboy/korean-pii-dataset`)는 CC BY 4.0이며 모델 저장소가 재배포하지 않는다.
- **일관성 100%**: 같은 문서는 언제나 같은 결과 → 실전 2에서 같은 사람이 같은 가명 토큰을 받는다.
- **멘토 holdout으로 재확인**: 처음 보는 문서 유형 19종에서도 개선 전 mask_rec 0.993(미탐 3개)으로 선정을 바꿀 이유가 없었다(⑥). 개선 후 0.995/1.000은 holdout에 맞춘 값이라 일반화 근거로 쓰지 않는다(⑧).
- **LLM은 RAG에서만**: 답변 생성에는 마스킹된 텍스트만 보낸다. 종합 단계의 출력 재검사에 LLM을 쓸지는 사내 LLM 여부에 따라 다시 검토한다.